One-Shot LLM: Engineering AI Requests from Dollars to Fractions of a Cent

version v0.13.1
date 8 Mar 2026
from Human (Dinis Cruz)
to Public. For publication on sgraph.ai/blog, LinkedIn
type Article. LLM architecture, prompt engineering, cost optimisation

---

I use AI in two completely different ways. The first way costs me $200 a month. The second way costs me fractions of a cent per request. And the second way is where I think the real future is.

Let me explain the difference, because I think most people are conflating two very different things when they talk about using LLMs in production.

The Two Workflows

Workflow 1: Let the platform manage it. I use Claude Code and Claude's web interface daily. I let it manage the conversation. I let it handle multi-turn dialogues, large contexts, and complex reasoning chains. I have 18 AI agents with different roles, and when I'm working with them through Claude, I don't micromanage the context. I let the platform optimise. This costs me $200 a month on the Max plan, and it's worth every penny.

Workflow 2: One-shot API requests where I control everything. This is what I want to talk about. When I call an LLM via API, every request is self-contained. No conversation history accumulates. I construct exactly what the model sees. I control the entire reality. And each request costs a fraction of a cent.

These are not the same thing. Workflow 1 is about leveraging a sophisticated platform that manages context for you. Workflow 2 is about engineering your own context management so precisely that you can use LLMs at a cost that approaches zero.

The sweet spot I'm excited about is Workflow 2. Because when a single LLM request costs you a penny, everything changes.

What a Penny Buys You

Let's do the maths. A capable model charges roughly $1 to $3 per million input tokens and $5 to $15 per million output tokens. That means if you send 50,000 tokens (which is a LOT of context, think multiple documents, code files, and a detailed question) and receive 2,000 tokens back, you're paying somewhere between 5 and 20 cents.

Now here's the thing. 50,000 tokens is what an unoptimised request looks like. Not because the developer is doing anything wrong, but because treating prompts as engineering artefacts (rather than chat messages) is a relatively new discipline. Most teams haven't invested in the non-functional requirements of their LLM usage: context management, model selection, prompt structure, cost tracking. When you send an entire file because you haven't built the tooling to extract the relevant section, or when you include conversation history that's irrelevant to the current question, or when you use a multimodal model for a text-only request, that's not laziness. That's the natural state before you start treating prompts as engineering tasks that deserve the same rigour as any other system design.

The good news is that once you do start treating them that way, the cost reduction is dramatic.

When I curate my context properly, most of my production one-shot requests are 5,000 to 15,000 tokens in and 1,000 to 2,000 tokens out. That puts each request at roughly one to five cents. Often less than a cent. But I also have exploratory requests that deliberately use 100,000 or 200,000 tokens of rich context with the most capable (and expensive) models, because sometimes you need to understand what's possible before you can compress it. Those might cost a dollar or two each. The difference is that the expensive ones are intentional investments that I save, analyse, and use to build the cheap ones.

And I'm not sacrificing quality. I'm often getting better results, because the model's attention isn't diluted across 40,000 tokens of irrelevant context. It's focused on exactly what matters.

Controlling the Reality

When I send a one-shot request via API, I control the model's entire universe. This is worth pausing on.

The model sees exactly what I put in the messages array. It doesn't see my filesystem. It doesn't see previous conversations. It doesn't see other projects. I am constructing the model's entire reality for each request. Everything it knows about the task, the context, the constraints, and the expected output is because I deliberately included it.

That constructed reality might include a system prompt that defines the agent's role. It might include context files: a project brief, an architecture document, a code snippet. It might include what I call a "constructed history," which is a hand-crafted series of user/assistant messages that prime the model's understanding. And then it includes the actual question.

Every element is there because I chose to include it. Everything the model doesn't need? Simply absent. Not hidden, not restricted. Absent.

Constructed History: Designing Memory

This is the part that surprises people. Sometimes I include a fake conversation history in my one-shot requests. Not a real conversation. A fabricated series of exchanges where I wrote both sides.

Why? Because LLMs are trained on conversations. They respond differently to a question that arrives cold versus one that arrives after a dialogue that built up context step by step.

If I want the model to understand a page structure before writing a transformation script, I don't dump the full HTML into a massive prompt. Instead, I construct a conversation where "the user" explains the page structure and "the assistant" acknowledges understanding it. By the time the model hits my actual question, it has already "understood" the context, because I told it what it understood.

I'm designing the model's memory, not letting it accumulate randomly. And the entire constructed conversation, including my question, still comes in under 15,000 tokens. Under five cents.

The Sandbox Is the Point

Here's something that gets overlooked: a one-shot API prompt is inherently sandboxed.

There is no execution happening on my local machine. The model can't run code, can't access files, can't make network calls. It receives text, it returns text. If the response contains malicious instructions, nothing happens, because there's nothing to execute them.

Compare this with an agent that has tool access: it can read files, write files, run shell commands, make HTTP requests. Every one of those capabilities is an attack surface. Every tool the agent can use is a tool that prompt injection can abuse.

Now, I'm not saying Workflow 1 (Claude Code with full access) is wrong. I use it daily and it's incredibly productive. But when I'm building for production, when I'm designing systems that will run unsupervised, when I'm constructing workflows that need to be predictable and auditable, the one-shot model is where I go. The worst case is a bad response that I don't use. Not a compromised system.

The Bundle: Version Control for Prompts

Over the past few months, I've developed a concept I call the "execution bundle." A bundle is a complete snapshot of everything that goes into a one-shot request, plus the response that came back.

A bundle contains: the system prompt, the context files (as references to stored documents), any constructed conversation history, the actual question, the model used, the response, and the metadata (token count, cost, duration, timestamp).

Bundles are saved to an encrypted vault. They're versioned. They're replayable. I can load a previous bundle, change one element, and re-execute. And they form a tree: when I fork from a previous bundle and try a different approach, the new bundle links to its parent, creating a branching history.

This is git for prompts. And because each bundle records its cost, I can see exactly how my prompt engineering is improving over time.

The Compression Path

Once a one-shot prompt works, the real fun starts. Can I achieve the same output with less context?

Now, if you're currently looking at LLM bills of thousands of dollars a month, the idea that individual requests should cost pennies probably sounds unrealistic. So let me walk through how you get there, because it's a progression, not a starting point.

I start with expensive requests. Sometimes a dollar or more for a single prompt. I'm exploring. I'm using the best, most expensive model available. I'm sending 200,000 tokens of context because I don't yet know which parts matter. I'm trying to understand the art of the possible. What can the model do with this task? What does a good output look like? These exploratory requests get saved as bundles, specifically so I can analyse them later and understand what actually contributed to the result.

This is the Genesis phase, and spending a dollar or two per request here is fine. In fact, it's necessary. You need to understand the full picture before you can compress it. I might do a handful of these on a new task type, and I save every single one.

Then I curate. Now that I know what works, I start removing context that didn't contribute. I replace the full document with a structural skeleton. I replace verbose instructions with a concise schema. I swap the expensive multimodal model for a text-only model (because if I'm not sending images, why am I paying for multimodal capability?). The request drops to 10,000 tokens. Five to ten cents.

Then I compress further. Pre-computed summaries instead of raw documents. Structured JSON schemas instead of natural language. The cheapest model that still produces acceptable results for this specific task. The request drops to 3,000 tokens. One cent. Maybe less.

And sometimes, at the end of this process, I realise I don't need the LLM at all. The prompt engineering process taught me enough about the task that I can write a deterministic script. Zero tokens. Zero cost.

The LLM helped me write the code that replaced it.

This is the Wardley Map of prompt engineering: Genesis ($1-2, exploratory, best models, save everything for analysis), Custom Built (10-20 cents, curated context, right-sized model), Product (1-5 cents, compressed, schema-driven), Commodity (zero, the LLM wrote its own replacement).

The point isn't that every request starts at a penny. The point is that every request should be on a path toward a penny. The Genesis requests that cost a dollar are investments in understanding. The Product requests that cost a cent are the return on that investment. And if a request has been running in production for weeks and still costs a dollar, that's the signal that nobody has done the engineering work to compress it.

What's Coming Next

Right now, my one-shot workflow handles individual requests beautifully. Each one is cheap, focused, and auditable. But I haven't yet built the orchestration layer that chains multiple one-shot requests into complex workflows with the same sophistication that Claude Code provides for Workflow 1.

That's next. The principle will be the same: each step in the chain is a one-shot request with curated context. The output of step 1 becomes part of the input for step 2. But every step is visible, every step has a cost, and every step can be optimised independently.

When that orchestration layer is ready, I expect to be able to run complex multi-agent workflows, the kind I currently do through Claude Code, at a fraction of the cost. Not because I'm using weaker models, but because every request in the chain is engineered to send only what's needed.

The Bellwether

Here's my rule of thumb: if an LLM API request that's been running for a while in a known workflow still costs you more than a few cents, you have an engineering opportunity. Not a model problem. Not a provider problem. An engineering opportunity.

It means the context hasn't been curated yet. It means nobody has asked "which parts of this prompt actually contribute to the output?" It means the model selection hasn't been optimised for this specific task. These are the non-functional requirements of LLM usage, and like all NFRs, they don't get addressed until someone decides they matter.

New, exploratory requests can and should cost more. A dollar or two for a Genesis prompt is an investment in understanding. But that investment should pay off. The same request, a week later, should cost a tenth of what it cost on day one. A month later, maybe a hundredth. And eventually, for some tasks, the cost should be zero because you've engineered the LLM out of the loop entirely.

The models are cheap enough now that a well-engineered request costs almost nothing. The question isn't "can I afford to use AI?" The question is "am I engineering my AI usage well enough that cost becomes irrelevant?"

When every production request costs a penny, you stop thinking about cost and start thinking about what to build.

This article is the third in a series on building production LLM workflows. Previous articles: "If You're Spending a Lot of Money on LLMs, You Have an Engineering Problem" and "How Do I Prove I Am Who I Am?"